

+ TEMA 09

HERRAMIENTAS

- Tipos de aprendizaje de modelos IA
- Entrenamiento de un modelo
- Predicción
- Ejercicio interactivo (Google Colab)





+ INTRODUCCIÓN **DE LA SESIÓN**

- + Los modelos de IA permiten automatizar decisiones y generar predicciones a partir de datos.
- + El desarrollo de modelos es el puente entre el análisis de datos y la solución inteligente.
- + Comprender cómo funcionan los modelos es clave antes de su implementación.
- + En esta sesión, aprenderemos los conceptos básicos del modelado IA y su aplicación práctica.



+

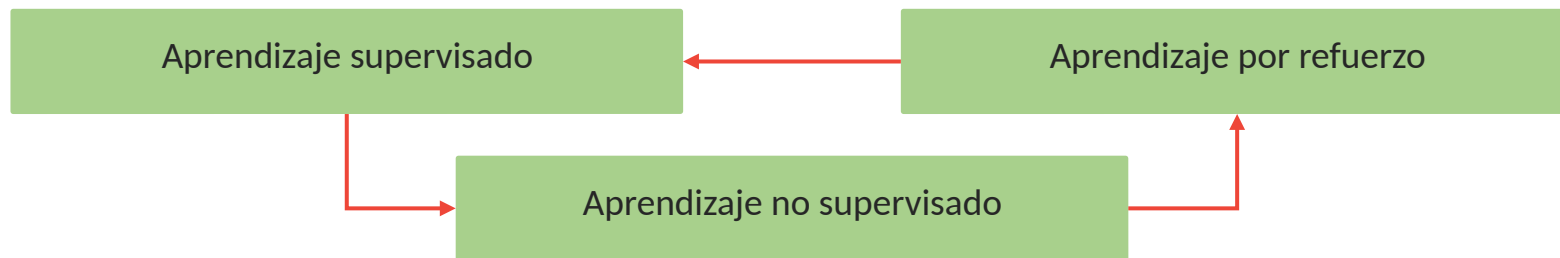
TIPOS DE APRENDIZAJE **DE MODELOS IA**

¿QUÉ ES EL APRENDIZAJE AUTOMÁTICO?

Es una rama de la IA que permite a las computadoras aprender patrones a partir de datos en lugar de ser programadas explícitamente.

Objetivo: crear modelos predictivos usando datos históricos para anticipar resultados futuros.

Los algoritmos de ML siguen distintos paradigmas:



APRENDIZAJE SUPERVISADO

Este modelo se entrena con datos etiquetados. El algoritmo aprende una función que mapea las entradas de las etiquetas de salida correctas.



Clasificación

Asignar categorías (por ejemplo, detectar spam).



Regresión

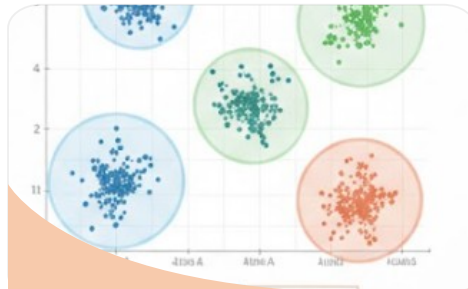
Predecir valores numéricos (por ejemplo, precio de una casa).

El aprendizaje supervisado es útil cuando disponemos de datos históricos con la respuesta correcta para cada caso.

APRENDIZAJE NO SUPERVISADO

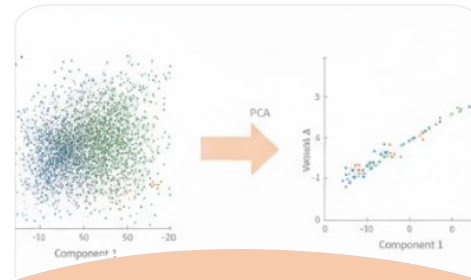
Trabaja con datos sin etiquetas, ya que el modelo actúa como un explorador. Busca estructuras ocultas por sí mismo.

Agrupar o resume información basándose en similitudes



Clustering

Agrupar clientes parecidos
(segmentación de mercado).



Reducción de dimensión

Simplificar muchas variables
complejas (ejemplo: PCA).

APRENDIZAJE POR REFUERZO

Este paradigma involucra a un agente que aprende a tomar decisiones mediante prueba y error en un entorno.

¿Cómo aprende?

- No usa datos etiquetados, sino experiencia.
- Recibe recompensas (si lo hace bien) o castigos (si lo hace mal).
- **Objetivo:** descubrir qué acciones dan la mayor recompensa a largo plazo.



COMPARATIVA DE PARADIGMAS

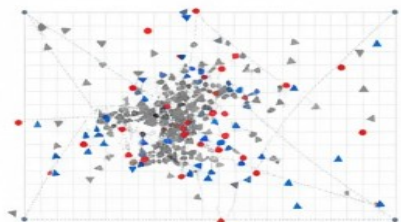
PARADIGMA	¿QUÉ NECESITA?	OBJETIVO PRINCIPAL	EJEMPLO
Supervisado	Datos etiquetados.	Predecir (función).	Diagnóstico médico.
No supervisado	Datos sin etiquetar.	Descubrir estructura.	Segmentar clientes.
Por refuerzo	Interacción (feedback).	Mejorar decisiones.	Aterrizaje de drones.

MÁS ALLA DE LO BÁSICO

1

Aprendizaje semisupervisado

Aprovecha un gran volumen de datos no etiquetados junto con pocos datos etiquetados, útil cuando el etiquetado es costoso.



2

Aprendizaje por transferencia

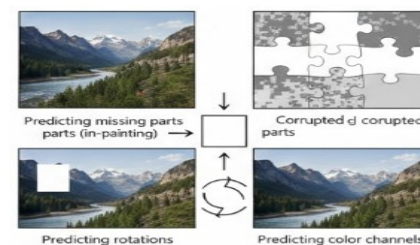
Consiste en reutilizar un modelo previamente entrenado y ajustarlo parcial o totalmente a una nueva tarea relacionada. Es común en deep learning cuando se dispone de pocos datos.



3

Aprendizaje autosupervisado

El modelo genera sus propias etiquetas a partir de los datos (por ejemplo, prediciendo partes faltantes de la entrada).



+

ENTRENAMIENTO
DE UN MODELO

EL CICLO DE UN PROYECTO DE ML

Fase 1: enfoque de fases (estructura lógica)

- **Objetivo:** definir métricas y establecer en una línea base (benchmark).
- **Datos:** recolección, limpieza y análisis exploratorio (detectar outliers, nulos).
- **Ingeniería:** selección y transformación de atributos relevantes.

Fase 2: modelado (el núcleo)

- **Entrenamiento:** dividir datos, elegir algoritmo y ajustar parámetros.
- **Validación:** evaluar métricas y realizar tuning de hiperparámetros iterativamente.

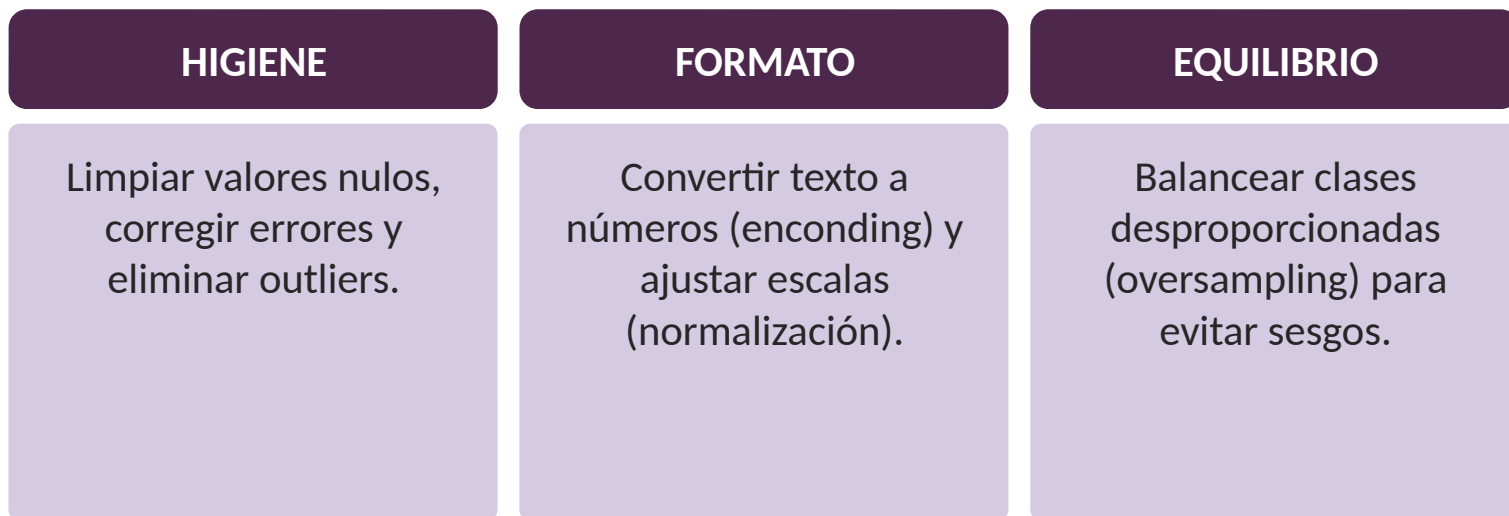
Fase 3: producción (el valor)

- **Despliegue:** integrar el mejor modelo en el sistema real.
- **Mantenimiento:** monitorear el desempeño y recalibrar si los datos cambian (data drift).



PREPROCESAMIENTO DE DATOS

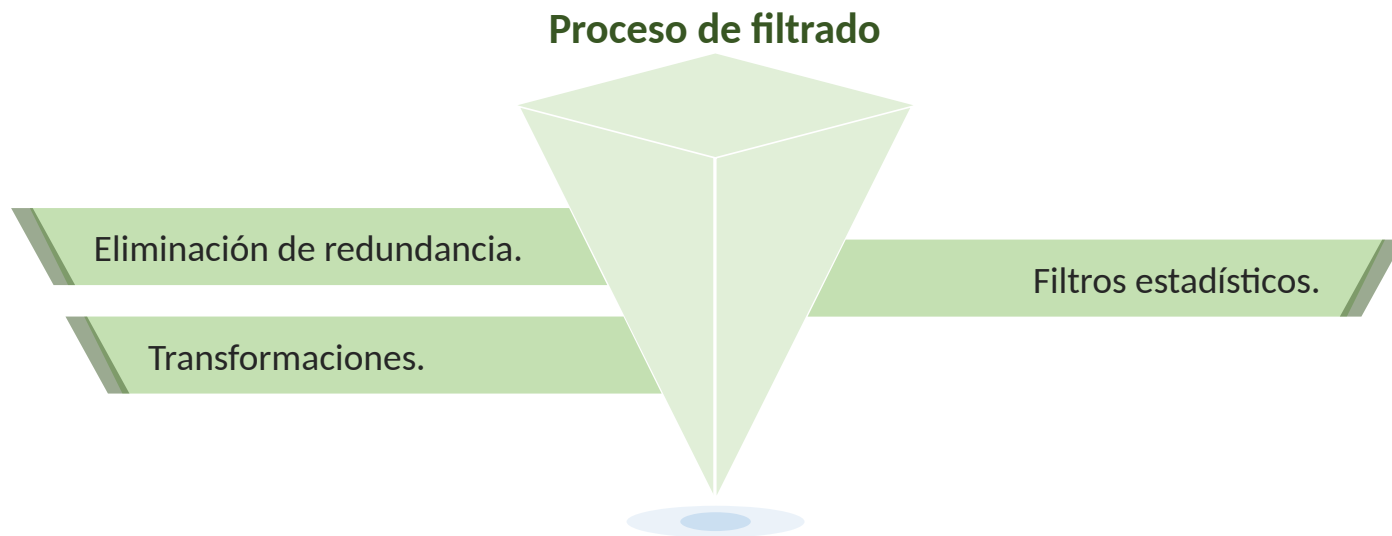
“Garbage in, garbage out”: la calidad del modelo depende directamente de los datos que le otorgamos.



Un buen preprocesamiento garantiza que los datos cumplan las suposiciones del algoritmo y mejora la eficiencia del entrenamiento.

SELECCIÓN DE ATRIBUTOS (CARACTERÍSTICAS)

Entrada: todas las variables crudas (ID, duplicados, ruido).



Salida: variables pertinentes.

Resultado: un modelo más rápido y preciso.

ELECCIÓN DEL MODELO Y ALGORITMO

MODELO / ALGORITMO	CARACTERÍSTICAS Y USO PRINCIPAL
Árbol de decisión	Manejan bien datos categóricos y destacan por ser interpretables (fáciles de explicar).
Bosques aleatorios (random forest)	Evolución de los árboles; suelen mejorar la precisión al combinar múltiples árboles.
SVM (máquinas de vector soporte)	Ideales para modelar fronteras complejas entre clases.
Redes Neuronales (incluyendo deep learning)	Capturan patrones muy complejos , pero requieren una gran cantidad de datos.
K-Nearest Neighbors (k-NN)	Algoritmo clásico basado en cercanía (mencionado como opción disponible).
Naive Bayes	Algoritmo probabilístico (mencionado como opción disponible).

ENTRENAMIENTO DEL MODELO

Predicción

El modelo recibe los datos de entrada (X) y genera una predicción ($\hat{y}$).

Medición del error

Una función de pérdida calcula qué tan alejada está la predicción del valor real.

Comparación

La predicción se compara con el valor real (y).

Ajuste

Se actualizan los parámetros internos del modelo para reducir el error; por ejemplo, mediante el algoritmo de descenso de gradiente.

EVALUACIÓN Y MÉTRICAS DE DESEMPEÑO

1

Conjunto validación / test

Reservamos un 20-30% de datos que el modelo nunca ha visto para simular la realidad (o usamos cross-validation).

2

Métricas para clasificación (sí / no)

- **Accuracy:** porcentaje de aciertos totales (cuidado: engañosa si las clases están desbalanceadas).
- **Precisión vs. recall:** miden la calidad de los aciertos y la capacidad de no olvidar ningún caso positivo
- **ROC/AUC:** capacidad de separar bien las clases (1.0 es perfecto).

3

Métricas para regresión (números)

Error cuadrático medio (MSE) y R2: miden qué tan lejos quedaron los números predichos de los reales.

VALIDACIÓN CRUZADA Y AJUSTO DE HIPERPARÁMETROS

EL PROBLEMA ¿FUE SUERTE O BUENO?



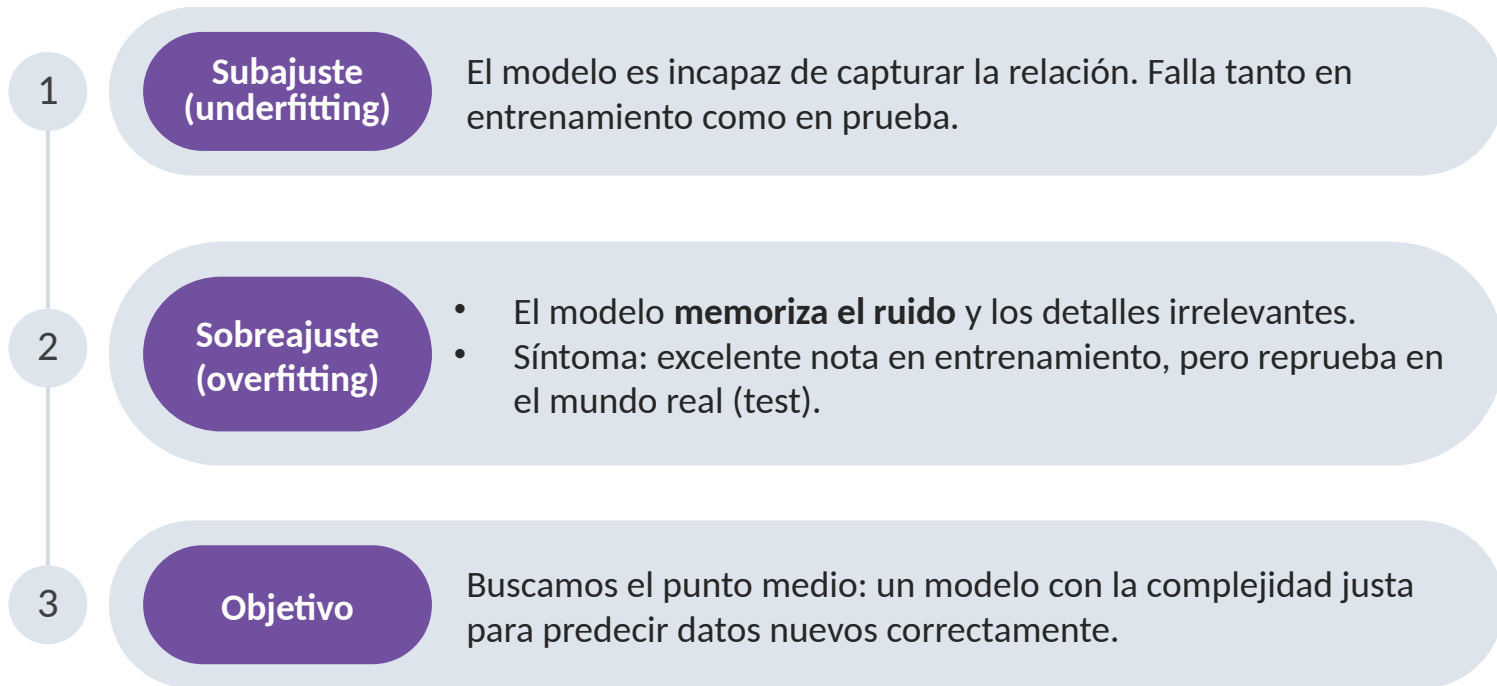
- Evaluar con una sola división de datos puede ser engañoso.
- **Solución: validación cruzada (k-fold).** Dividimos los datos en k partes y probamos k veces rotando. El resultado es el promedio (más robusto).



EL PROBLEMA ¿QUÉ CONFIGURACIÓN USO?

- Los algoritmos tienen “perillas” externas (hiperparámetros) que no aprenden solos (ejemplo: profundidad del árbol, k vecinos).
- **Solución: tuning (ajuste)**
 - **Grind search:** probar todas las combinaciones (lento pero seguro).
 - **Random search:** probar al azar (más rápido y eficiente).

EVITAR SOBREAJUSTE (OVERFITTING)



+
PREDICCIÓN

USANDO EL MODELO ENTRENADO - PREDICCIÓN

Ingreso de nuevos datos

El usuario o sistema introduce información "fresca" (ejemplo: síntomas de un nuevo paciente).

Preprocesamiento (¡CRÍTICO!)

Los datos nuevos deben sufrir exactamente las mismas transformaciones que los de entrenamiento para que el modelo los entienda.



Inferencia

El modelo aplica lo aprendido y genera una salida bruta (probabilidad o valor numérico).

Interpretación (postproceso)

Traducir la salida bruta a algo útil (ejemplo: si la probabilidad es mayor a 0.5, el riesgo es alto).

INTERPRETANDO LOS RESULTADOS

Matriz de confusión

Desglosa los errores:
¿estamos confundiendo
sistemáticamente una clase
con otra?

Importancia de atributos

Pregunta clave: ¿qué
variables pesaron más en la
decisión? Ejemplo: en
random forest, ver qué
síntoma define la
enfermedad.



Análisis de errores individuales

Revisar manualmente los
casos fallidos para encontrar
patrones extraños.

Línea base (benchmark)

Comparar con la realidad: si el
modelo acierta 85% y un
humano 80%, es útil. Si el
humano acierta 95%, el
modelo no sirve.

INTELIGENCIA ARTIFICIAL EXPLICABLE (XAI) E INTERPRETABILIDAD

01 ● EL DESAFÍO

Modelos potentes como el deep learning son opacos: sabemos que predicen, pero no por qué.

02 ● LA SOLUCIÓN (XAI)

- Conjunto de técnicas para hacer visible el razonamiento de la IA.
- Objetivo: responder “¿Por qué el modelo tomó esta decisión?” y “¿Cuánto podría fallar?”.

03 ● ¿POR QUÉ IMPORTA?

- Confianza: un médico necesita saber qué vio el algoritmo en una radiografía para confiar en el diagnóstico.
- Ética: ayuda a detectar si el modelo está discriminando por género o raza (sesgos ocultos).



CASO DE USO

El reto: predecir si un paciente tiene diabetes (sí/no) usando sus datos clínicos.

La preparación

- Limpiamos valores faltantes de glucosa.
- Escalamos variables (IMC, presión) para que sean comparables.



El modelo: entrenamos una regresión logística (tras ajustar hiperparámetros).

En acción: una app alerta al médico y usa SHAP para explicarle: "Riesgo alto por glucosa elevada".

El resultado clave: logramos un recall del 90%.

- ¿Por qué? Priorizamos identificar a la mayoría de los pacientes afectados para minimizar los falsos negativos.

+

EJERCICIO INTERACTIVO
(GOOGLE COLAB)

INTRODUCCIÓN A LA PRÁCTICA EN COLAB

01 ¿QUÉ ES?

Es un entorno de Jupyter Notebooks en la nube que permite escribir y ejecutar código Python directamente desde el navegador, sin necesidad de instalar programas en la computadora.

02 ¿POR QUÉ LO USAMOS?

- Sin instalaciones: no necesitas configurar Python en tu PC.
- Todo listo: librerías clave como pandas y scikit-learn vienen preinstaladas.
- Potencia: acceso gratuito a GPU para acelerar el cálculo.

03 EL EJERCICIO DE HOY

- Replicaremos el flujo completo: carga, sigue limpieza, se pasa al entrenamiento y al final la predicción.



PASO 1 - IMPORTACIÓN DE LIBRERÍA Y DATOS

Importar librerías: cargamos las herramientas necesarias. Pandas para datos, NumPy para números y scikit-learn para IA.

```
>>> import pandas as pd
... import numpy as np
... from sklearn.model_selection import train_test_split
... from sklearn.preprocessing import StandardScaler
... from sklearn.tree import DecisionTreeClassifier
... from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
```

Carga el dataset (por ejemplo, iris): usamos un dataset clásico de flores incluido en la librería.

```
>>> from sklearn.datasets import load_iris
... iris = load_iris()
... X = pd.DataFrame(iris.data, columns=iris.feature_names)
... y = pd.Series(iris.target) # especies de iris codificadas 0,1,2
```

PASO 2 - PREPROCESAMIENTO DE DATOS:

División de datos (train/test split): separamos el 20% de los datos para el "examen final" (test) y usamos el 80% para estudiar (train).

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42)|
```

Normalización (escalado): como los pétalos y sépalos pueden tener magnitudes distintas, los estandarizamos para ayudar al modelo.

```
scaler = StandardScaler()  
  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test) # usar misma transformación del train
```

PASO 3 - ENTRENAMIENTO DEL MODELO

Inicialización (configurar las reglas)

- Creamos un árbol de decisión vacío.
- `max_depth=3`: limitamos la profundidad para que el árbol sea simple y fácil de leer (hiperparámetro).

```
clf = DecisionTreeClassifier(max_depth=3, random_state=42)
```

Entrenamiento (aprender patrones)

- El método `.fit()` es donde ocurre la magia: el modelo estudia los datos (x) y sus respuestas (y).

```
clf.fit(X_train_scaled, y_train)
```

Inspección

- Podemos ver que aprendió consultando `clf.feature_importances_`.

PASO 4 - EVALUACIÓN DEL MODELO

Generar predicciones: el modelo usa los datos de prueba (que nunca vio antes) para intentar adivinar su especie.

```
>>> y_pred = clf.predict(X_test_scaled)
```

El código: generamos la matriz para ver exactamente dónde se equivocó el modelo.

```
>>> cm = confusion_matrix(y_test, y_pred)
... print("Confusion Matrix:\n", cm)
```

Calcular puntajes: comparamos predicción vs. realidad. Nota: usamos **average='macro'** porque hay 3 clases de flores (setosa, versicolor, virginica).

```
>>> acc = accuracy_score(y_test, y_pred)
... prec = precision_score(y_test, y_pred, average='macro')
... rec = recall_score(y_test, y_pred, average='macro')
... f1 = f1_score(y_test, y_pred, average='macro')
...
... print(f"Accuracy: {acc:.2f}, Precision: {prec:.2f}, Recall: {rec:.2f}, F1: {f1:.2f}")
```

PASO 5 - PREDICCIÓN CON NUEVOS DATOS

```
>>> # 1. Llega un dato nuevo (sépalos y pétalos)
... new_sample = np.array([[5.0, 3.5, 1.3, 0.3]])
...
... # 2. Lo estandarizamos (usando el scaler ya entrenado)
... new_sample_scaled = scaler.transform(new_sample)
...
... # 3. Predecimos la clase
... pred_clase = clf.predict(new_sample_scaled)
... print("Predicción de especie:", iris.target_names[pred_clase])
...
... # 4. (Opcional) Vemos la confianza
... probas = clf.predict_proba(new_sample_scaled)
... print("Probabilidades:", probas)
```

Interpretación

- Si obtienes [0.98, 0.02, 0.00], significa que el modelo está 98% seguro de que es una setosa (clase 0).

PASO 6 - INTERPRETABILIDAD Y VISUALIZACIÓN EN LA PRÁCTICA

El código de la interpretación: “Con estas líneas, el modelo nos confiesa sus secretos”.

```
>>> # 1. Reglas escritas
... from sklearn.tree import export_text
... rules = export_text(clf, feature_names=list(X.columns))
... print(rules)
...
... # 2. Diagrama visual
... from sklearn import tree
... import matplotlib.pyplot as plt
...
... plt.figure(figsize=(8,6))
... tree.plot_tree(clf, feature_names=iris.feature_names,
... class_names=iris.target_names, filled=True)
... plt.show()
```

¿Qué veremos?

- Texto: una lista jerárquica tipo |--- **petal width <= 0.75: class: setosa.**
- Gráfico: nodos de colores donde cada tono es una especie de flor distinta.

PASO 7 - EXPERIMENTACIÓN ADICIONAL



Ajustar hiperparámetros

Usar GridSearchCV de sklearn para encontrar automáticamente el mejor `max_depth` u otros parámetros para el árbol de decisión.

Probar otros modelos

Usar `RandomForestClassifier` o `KNeighborsClassifier` de sklearn, y ver si mejora las métricas en el conjunto de prueba.



Trabajar con datos

Cargar un dataset diferente (quizá datos reales de UCI Machine Learning Repository) para resolver otro problema.

Visualización de resultados amplia

Generar curvas ROC usando `sklearn.metrics.roc_curve` y `auc` si es binario, o gráficos de importancia de características.



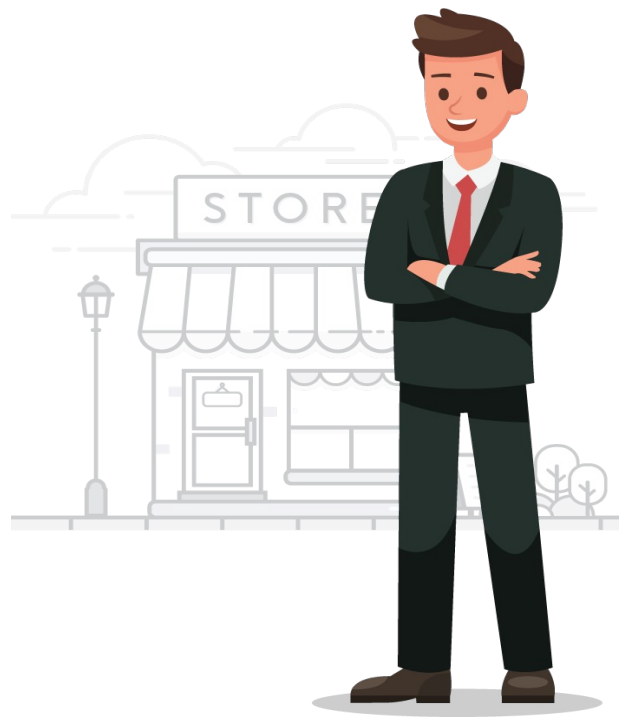
CONCLUSIÓN DE LA PRÁCTICA

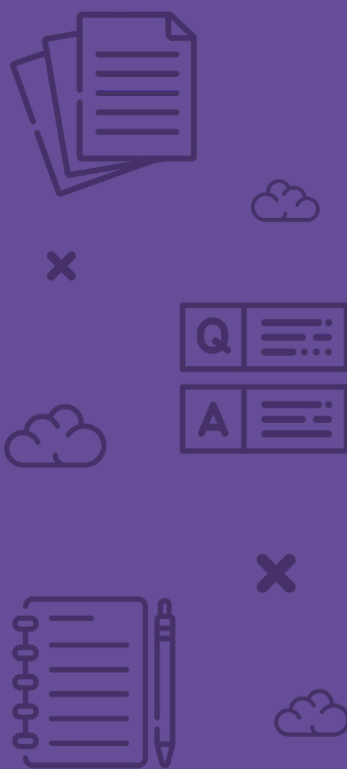
- **Poder de las herramientas:** vimos cómo scikit-learn y pandas simplifican tareas matemáticas complejas en pocas líneas de código.
- **Conexión total:** comprendimos cómo se enlazan los conceptos teóricos (supervisado, preprocesamiento) con la ejecución real (fit, predict, evaluate).
- **Más allá del dato:** no solo calculamos métricas; interpretamos el modelo (el árbol) para entender el "porqué" de sus decisiones.
- **Base sólida:** este flujo de trabajo es el estándar que usarás en casi cualquier proyecto futuro.



REFLEXIÓN FINAL

- Hemos visto cómo la IA combina algoritmos teóricos con aplicaciones prácticas en el mundo real.
- Exploramos cómo los tres paradigmas (supervisado, no supervisado, refuerzo) se adaptan a problemas muy distintos.
- Confirmamos que el éxito de un proyecto no depende solo del código, sino de la calidad de los datos y una evaluación rigurosa.
- Demostramos que, con herramientas modernas (Colab, Sklearn), implementar modelos útiles es posible con pocas líneas de código.



A collection of white line-art icons on a purple background. At the top left is a stack of three papers. Below it is a small 'x' icon. To the right of the 'x' is a cloud icon. Below the cloud is a Q&A icon consisting of two boxes, the top one labeled 'Q' and the bottom one labeled 'A', each with three horizontal lines representing text. To the left of the Q&A icon is another cloud icon. At the bottom left is a notepad icon with a pencil. To its right is a small 'x' icon. At the bottom right is a cloud icon.

+ **CONCLUSIONES**

+ CONCLUSIONES

- + Los modelos de IA permiten transformar datos en predicciones útiles.
- + Comprender los tipos de aprendizaje ayuda a elegir el enfoque correcto.
- + El entrenamiento es una etapa crítica del proceso de modelado.
- + Las predicciones generan valor cuando están alineadas a un problema real.
- + Esta sesión sienta las bases para la selección y evaluación de modelos en las siguientes clases.





x



x



+

BIBLIOGRAFÍA **MÁS REFERENCIAS**

- + Forero-Corba, W., & Bennasar, F. N. (2024). *Techniques and applications of Machine Learning and Artificial Intelligence in education: a systematic review*. RIED-Revista Iberoamericana de Educación a Distancia, 27(1)
- + Jamarani, A., Haddadi, S., Sarvizadeh, R., Haghi Kashani, M., Akbari, M., & Moradi, S. (2024). *Big data and predictive analytics: A systematic review of applications*. Artificial Intelligence Review, 57(7), 176
- + Mukhamediev, R. I., Popova, Y., Kuchin, Y., Zaitseva, E., Kalimoldayev, A., Symagulov, A., ... & Yelis, M. (2022). *Review of artificial intelligence and machine learning technologies: classification, restrictions, opportunities and challenges*. Mathematics, 10(15), 2552.
- + Usuga, I. (2023). *Preparación de Datos Para Aprendizaje Automático*.
- + Valverde Gómez, D. (2024). *Etapas de un proyecto de Aprendizaje Automático. Clasificación de incidencias*.



ISIL+